Federated Learning-based Intrusion Detection System for SDN-enabled IoT Networks

Aaron B Ajay
1BM23CS003

Aaryan Prakash
1BM23CS006

Amal Roy
1BM23CS025

Amogh Krishna J S
1BM23CS029

Guide: Mr. Syed Akram
Assistant Professor
Department of Computer Science and Engineering
B.M.S. College of Engineering

Outline

1. Sustainable Development Goals
2. Problem Statement and Objectives
3. High Level Design
4. Detailed Design
5. Implementation
6. Results: Datasets, Graphs, Tables and Output Snapshots
7. Conclusion and Future Enhancements
8. Papers Published
9. References

Sustainable Development Goals (SDGs) Addressed

9 SDG 9: Industry, Innovation and Infrastructure

- Strengthens the security of network infrastructure that IIoT systems depend on: manufacturing, power and transport.
- Detects attacks, resists poisoned training and blocks attackers through SDN flow rules, which supports smart systems.

16 SDG 16: Peace, Justice and Strong Institutions

- Strengthens cybersecurity and protects sensitive digital data.
- Raw traffic never leaves a site during training; only model weights and capped alerts are shared.
- Attacks are detected and blocked, which supports secure and resilient digital systems.

Problem Statement

IoT and IIoT networks managed with SDN face floods, scans, web attacks, malware and man-in-the-middle attacks, while their devices run with little host security. A centralised intrusion detection system needs all raw traffic in one place, which raises privacy, bandwidth and scalability problems.

Federated learning keeps traffic on each site, but it opens new weaknesses: a few poisoned client updates can ruin the shared model, and a classifier trained on known labels cannot recognise an attack it has never seen. Reported accuracy on the Edge-IIoTset data is also often inflated by random packet splits and by errors in the released CSV, and a detection rarely changes what the network does.

Therefore, there is a need for a federated IDS that resists poisoned updates, detects unseen attacks, is measured on a strict test, and turns detections into enforced network rules.

Privacy and scale
Raw traffic cannot be pooled centrally

Poisoned updates
30% bad clients broke plain FedAvg

Unseen attacks
A classifier only knows its labels

Inflated results
Random splits leak connections

No enforcement
Detection does not block traffic

Objectives

1 Malicious Client Detection and Filtering

Develop a trust-based filter on the server that identifies and excludes compromised clients from federated aggregation.

2 Efficient, Privacy-Aware FL Design

Build a lightweight federated framework in which raw traffic stays on each client, and link its verdicts to enforced SDN rules.

3 Model Optimization for Scalability

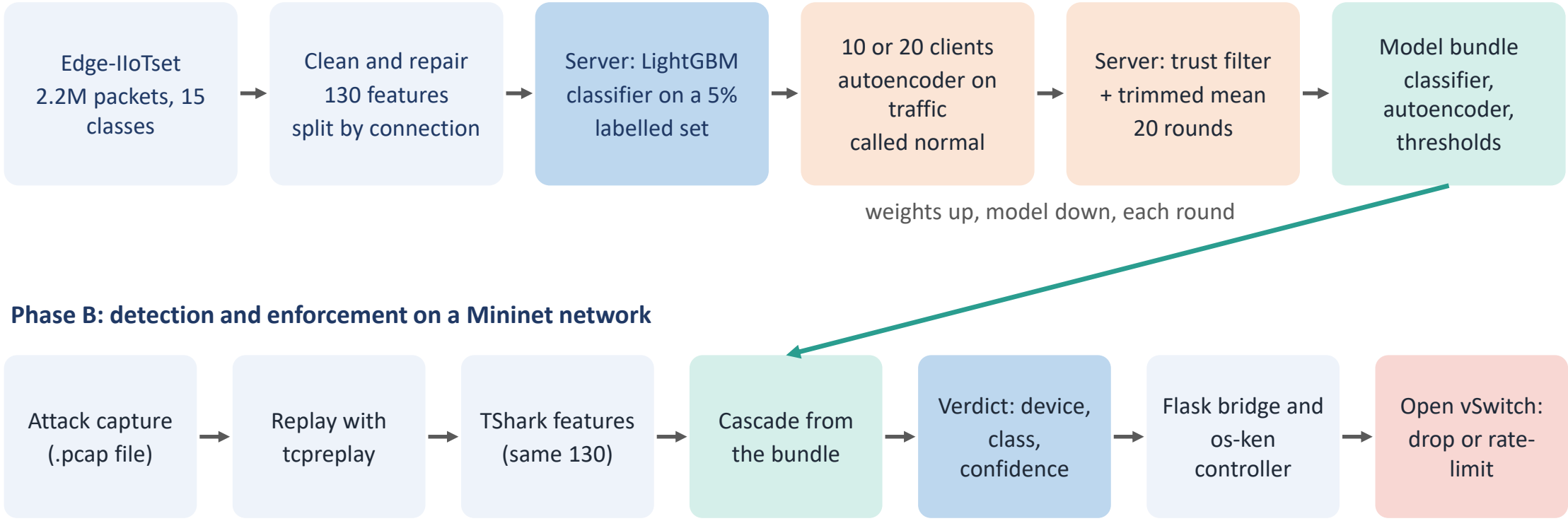
Apply weight pruning to reduce the computational and communication load on resource-constrained IoT devices.

4 Semi-Supervised and Anomaly-Based Detection

Add an autoencoder backstop to a boosting classifier so that attacks never seen in training are still flagged.

High Level Design

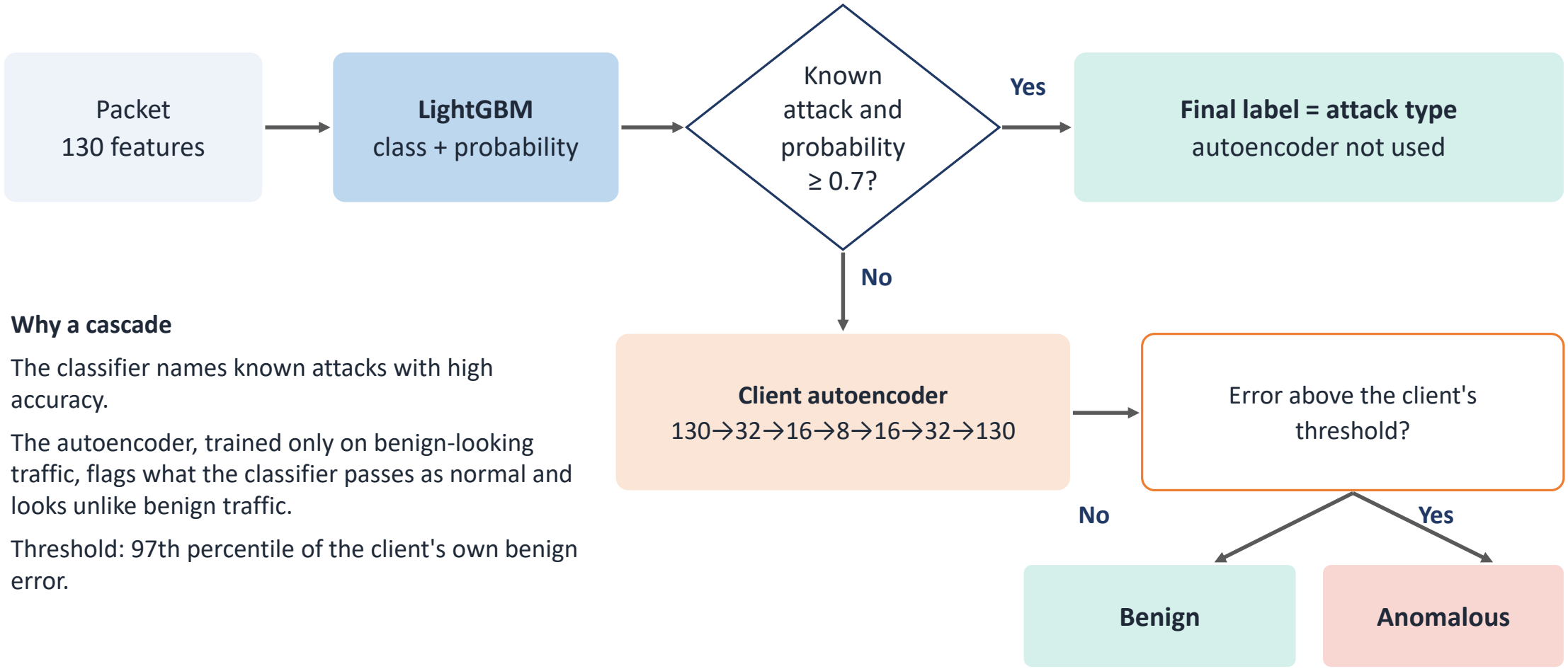
Phase A: federated training (offline)



Phase B: detection and enforcement on a Mininet network

Block a device when confidence is at least 0.85, **rate-limit** it at 0.5 or more. Phase B classifies a whole capture, not a live packet stream.

Detailed Design: Two-Stage Cascade



Why a cascade

The classifier names known attacks with high accuracy.

The autoencoder, trained only on benign-looking traffic, flags what the classifier passes as normal and looks unlike benign traffic.

Threshold: 97th percentile of the client's own benign error.

Detailed Design: One Federated Round

1 Broadcast

The server sends the global autoencoder and the classifier to every client.

2 Local training

Each client drops traffic the classifier already calls a confident attack and trains the autoencoder on the rest for 5 epochs.

3 Upload

Clients send the weight change, an error histogram and at most 50 confirmed alerts. Raw traffic stays local.

4 Trust filter

Exclude updates that are cosine or MAD outliers against the median update, point away from it, or come from clients with running trust below 0.5.

5 Aggregate

Clip update norms to 2 times the median, then average with a 15% trimmed mean to form the new global autoencoder.

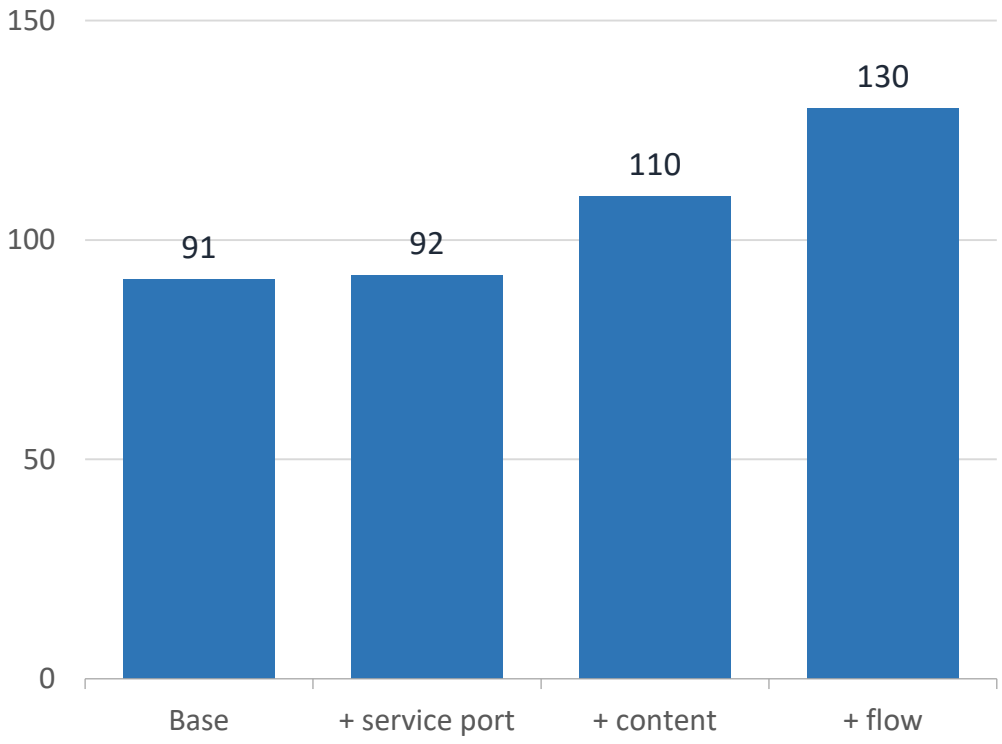
6 Boost update

Every 3rd round the server adds 50 trees per class, trained on the calibration set plus confirmed alerts.

Repeated for 20 rounds. Orange steps are the poisoning defence.

Detailed Design: Features and Data Split

Features per packet (cumulative)



Content: 18 payload and HTTP numbers. Flow: 20 per-connection counts. IP addresses and times only group packets and are never inputs.

Split by connection

Packets of one connection (IP pair, protocol, ports; 60 s idle or 120 s active timeout) stay together: 342,814 connections, so no connection is in both training and test.

Server calibration set

About 5% of rows, labelled, bootstraps the LightGBM classifier. The rest goes to the clients.

Non-IID clients

A Dirichlet(0.3) draw gives each of 10 or 20 clients a very different class mix. Per client: 15% test, 15% benign validation, rest training.

Implementation: Tools and Environment

Component	Used for	Version
Python	All code	3.14.4
LightGBM	Boosting classifier	4.7.0
PyTorch (CPU)	Autoencoder	2.14.0
Flower	Federated learning server and clients	1.38.0
scikit-learn, pandas, NumPy	Splits, metrics, data handling	1.9.1, 3.0.6, 2.5.3
Mininet, Open vSwitch	Emulated network and switch	2.3.0, 3.7.1
os-ken, Flask	OpenFlow controller and REST bridge	4.2.2, 3.1.3
TShark	Feature extraction from captures	4.6.4
Streamlit	Live dashboard	1.64.0

All training, results and Phase B ran inside the VM; only text results left it.

244

automated tests passed, 1 skipped

20 vCPU

VMware Ubuntu 26.04 LTS virtual machine with 48 GB RAM

~16 GB

data, run logs and saved models on disk

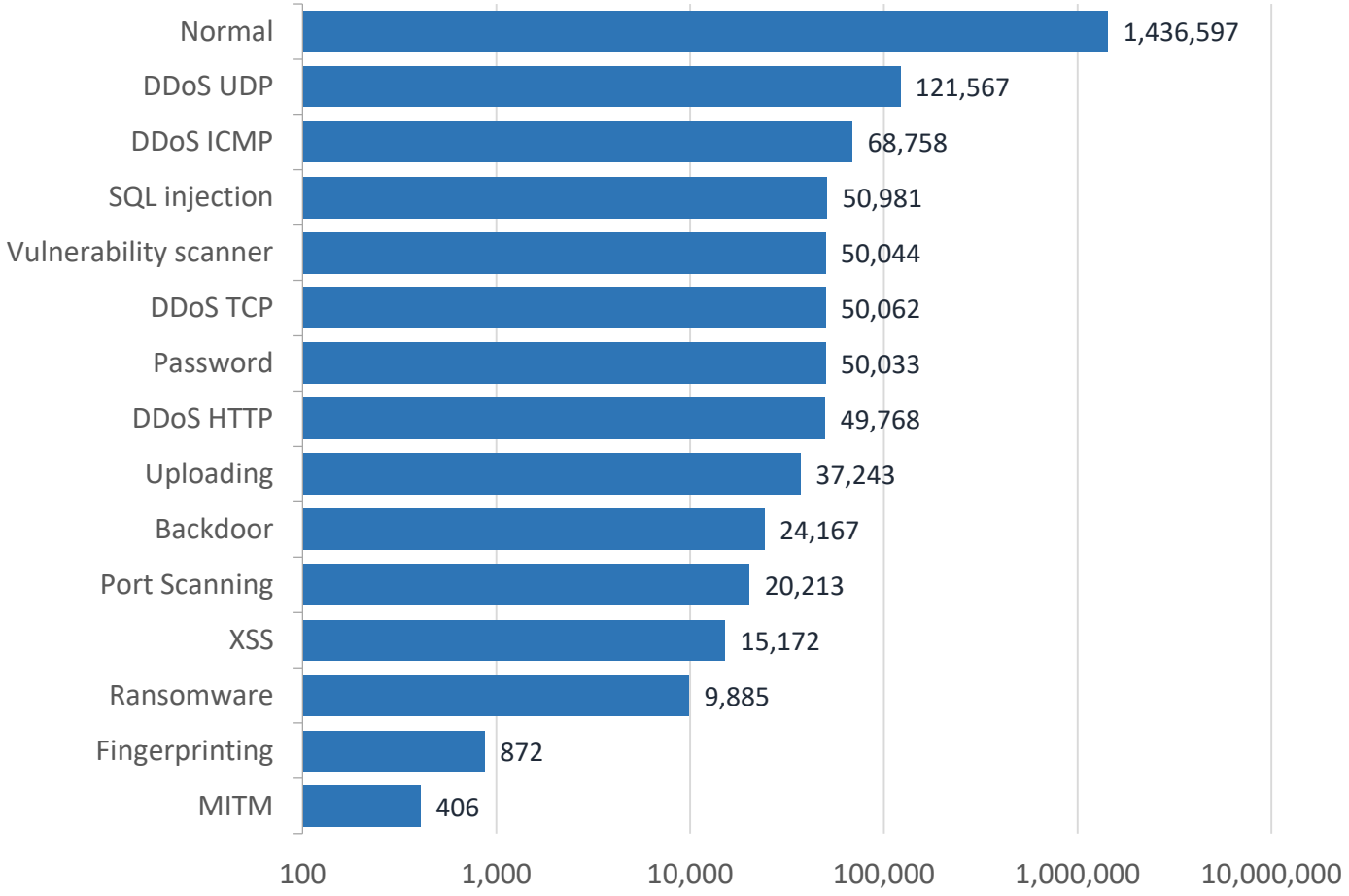
Implementation: Project Timeline



2026. Phases 1 to 3: research. Phases 4 to 6: build. Later phases: evaluation, enforcement and write-up.

Results: Dataset Used

Packets per class after cleaning (log scale)



1,985,768

unique packets from 2,219,201 raw rows

15 classes

normal traffic plus 14 attack types

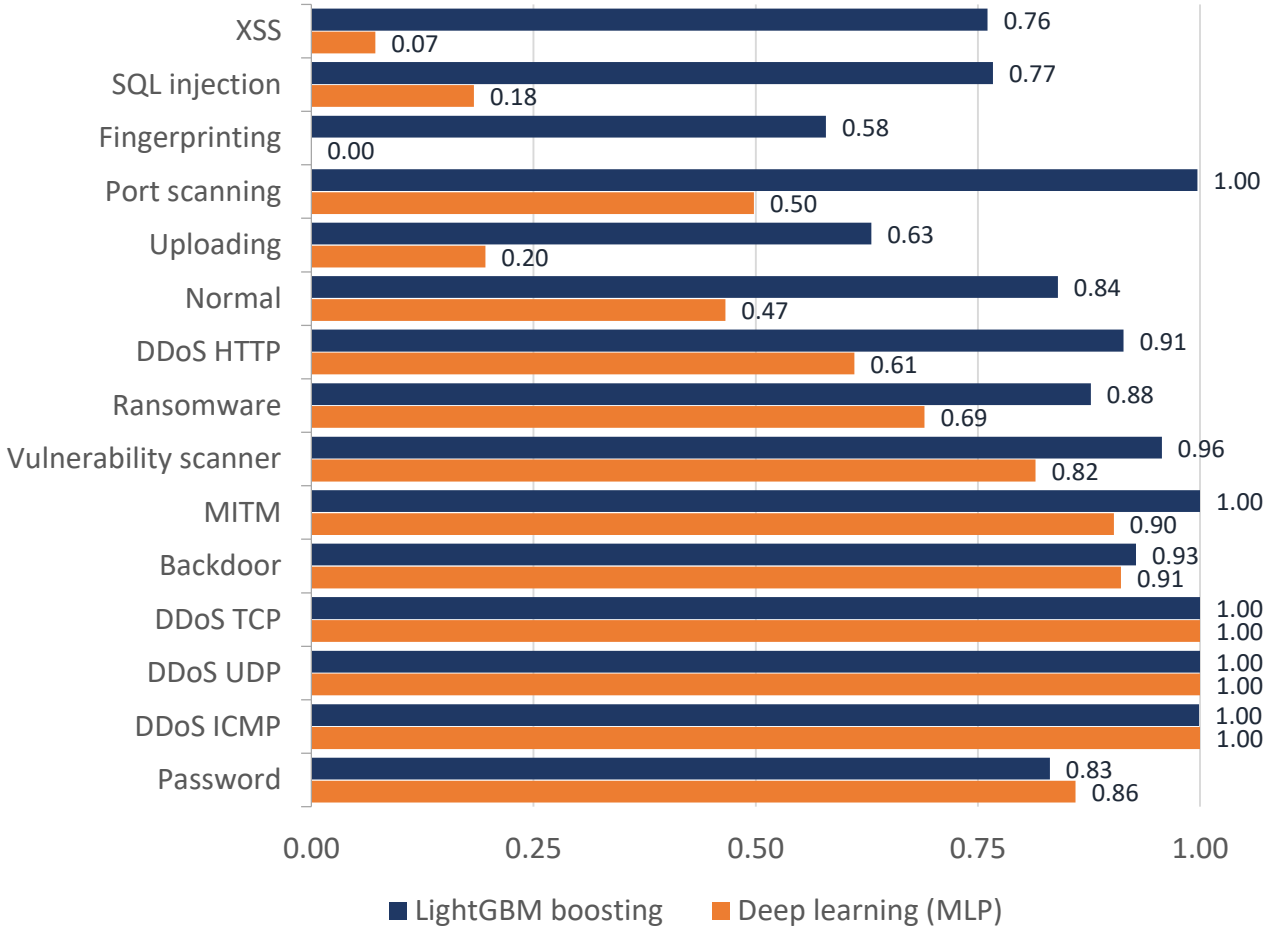
342,814

connections, kept whole in every split

MITM is a single connection (406 packets), so it cannot be split.

Results: Boosting vs Deep Learning

Recall per class (same 500,000-row sample)



0.887 vs 0.622

macro F1, boosting vs deep learning

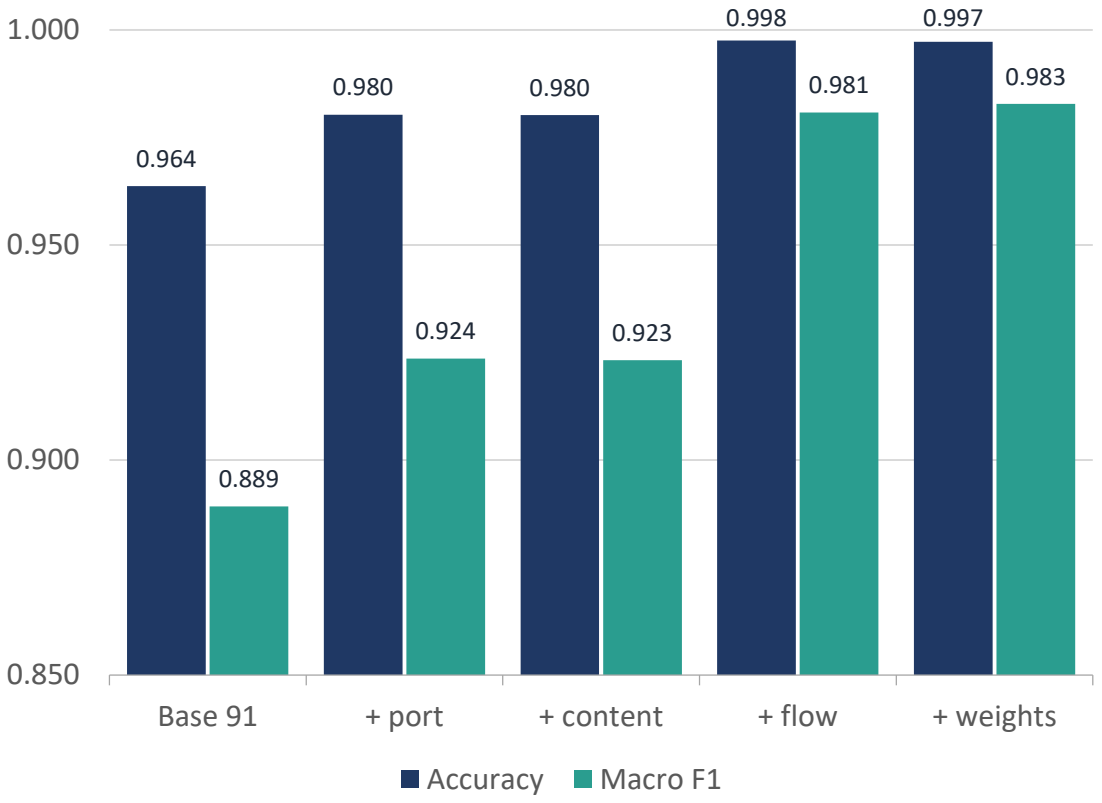
0.891 vs 0.678

accuracy, boosting vs deep learning

- The gap is largest on application-layer attacks that resemble normal web traffic (XSS, SQL injection, Uploading).
- On flood attacks both models are close to perfect.
- This supports boosting as the first stage.

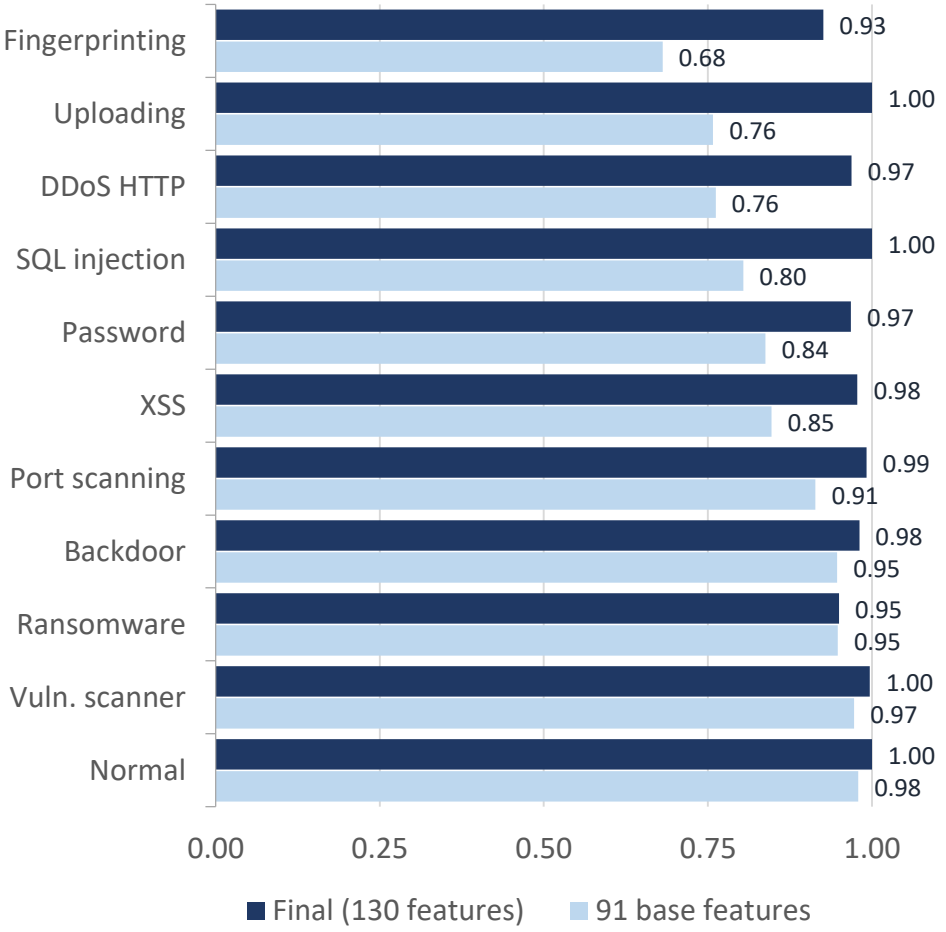
Results: Effect of the New Features

Held-out connections, as features are added

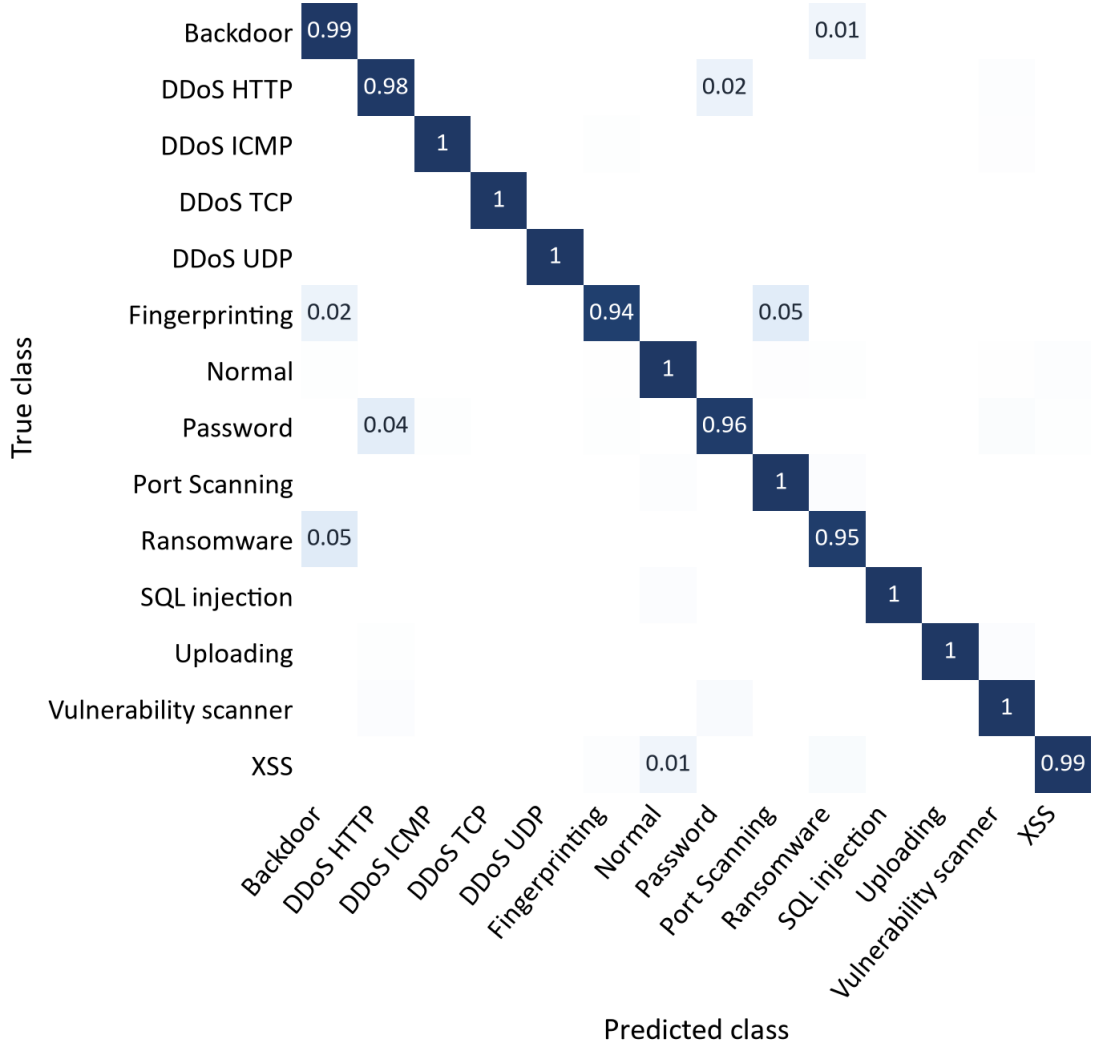


Vertical axis starts at 0.85. Final: 99.72% accuracy, 0.983 macro F1 on 397,073 unseen packets.

F1 per class: 91 features vs final



Results: Confusion Matrix of the Final Classifier



99.72%

accuracy on 397,073 packets from connections never seen in training

0.983

macro F1 across the 14 classes in the test split

- **Largest remaining errors:** Password called DDoS HTTP (4%), DDoS HTTP called Password (2%). Both are HTTP requests to the same web server.
- **Ransomware called Backdoor** (5%): both use port 4321.
- MITM is not in the matrix: it is one ARP connection.

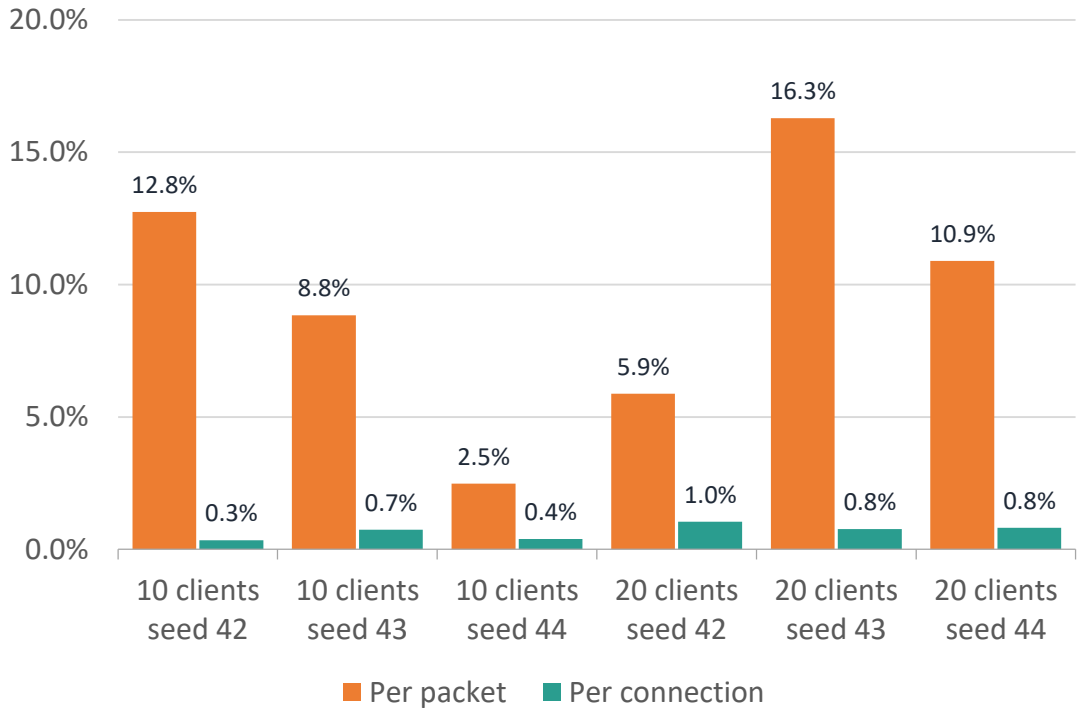
Shading is stretched so small errors show; each cell is the share of its true class.

Results: Federated Training on the Full Dataset

Attack macro-recall (cascade)



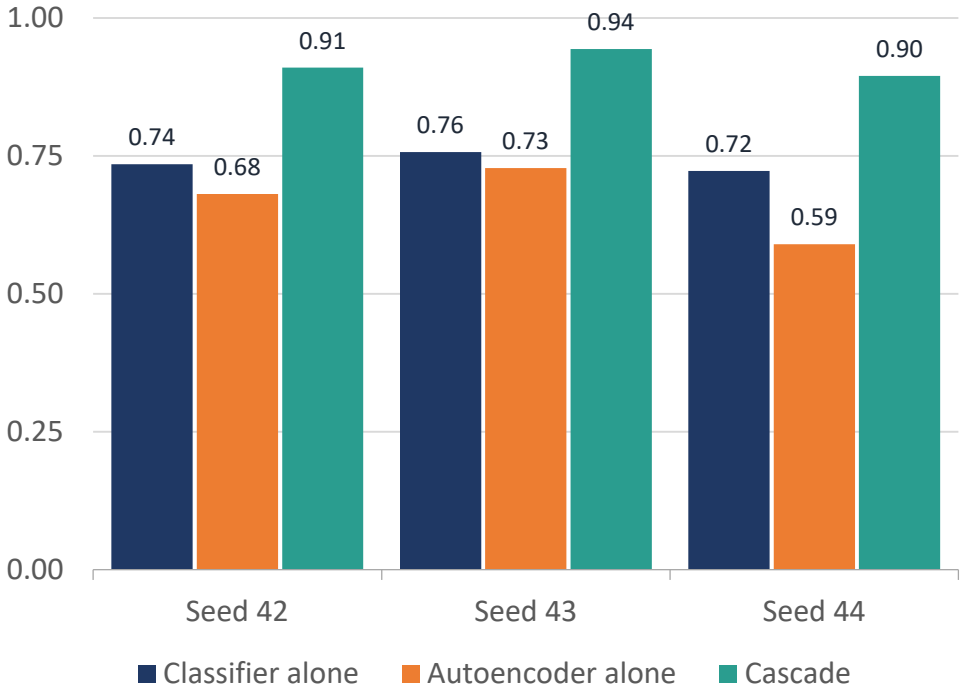
False alarms on normal traffic



1.89 million training rows, 20 rounds, one server and 10 or 20 client processes. Known attacks are caught in all six runs. Packet-level false alarms are 2.5% to 16.3%, but per connection only 0.34% to 1.04%, because the false alarms cluster in a few connections. Vertical axis of the left chart starts at 0.99.

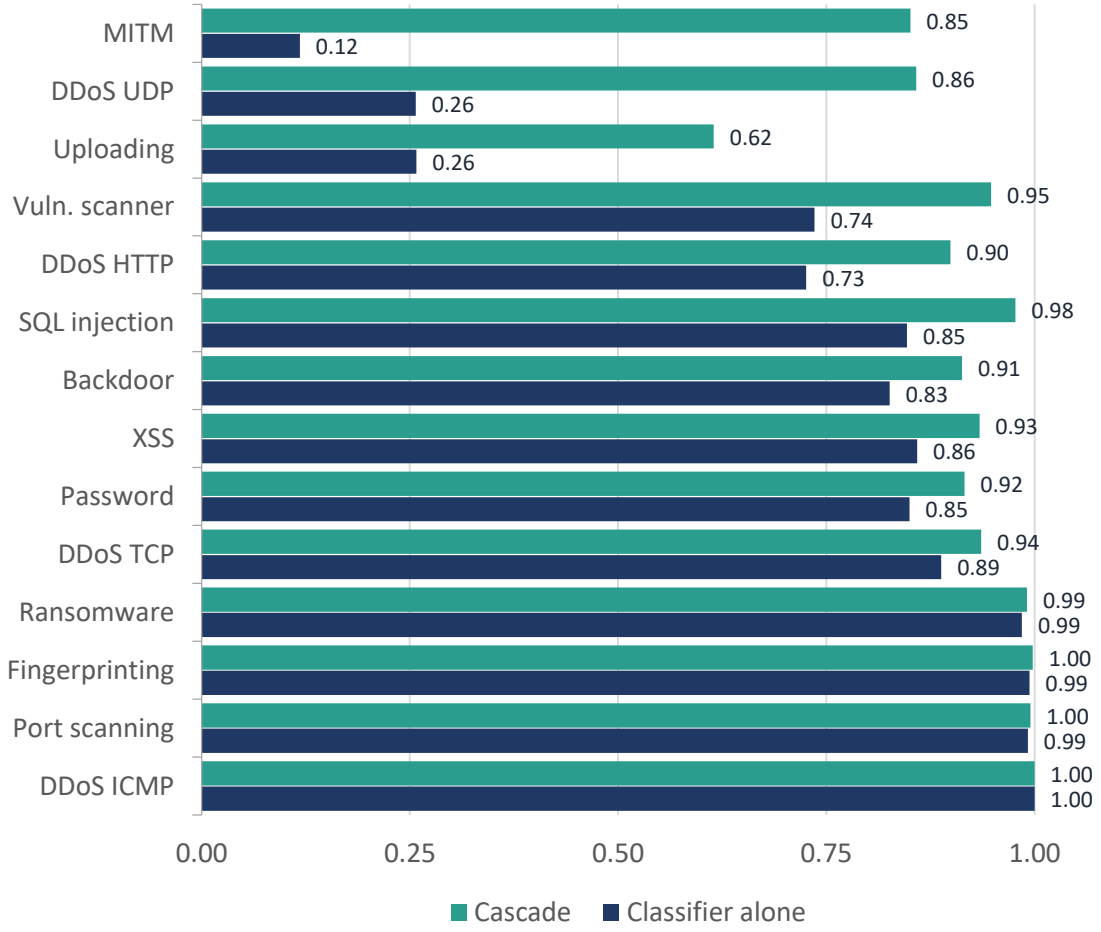
Results: Detecting Unseen (Zero-Day) Attacks

Mean detection of the held-out class

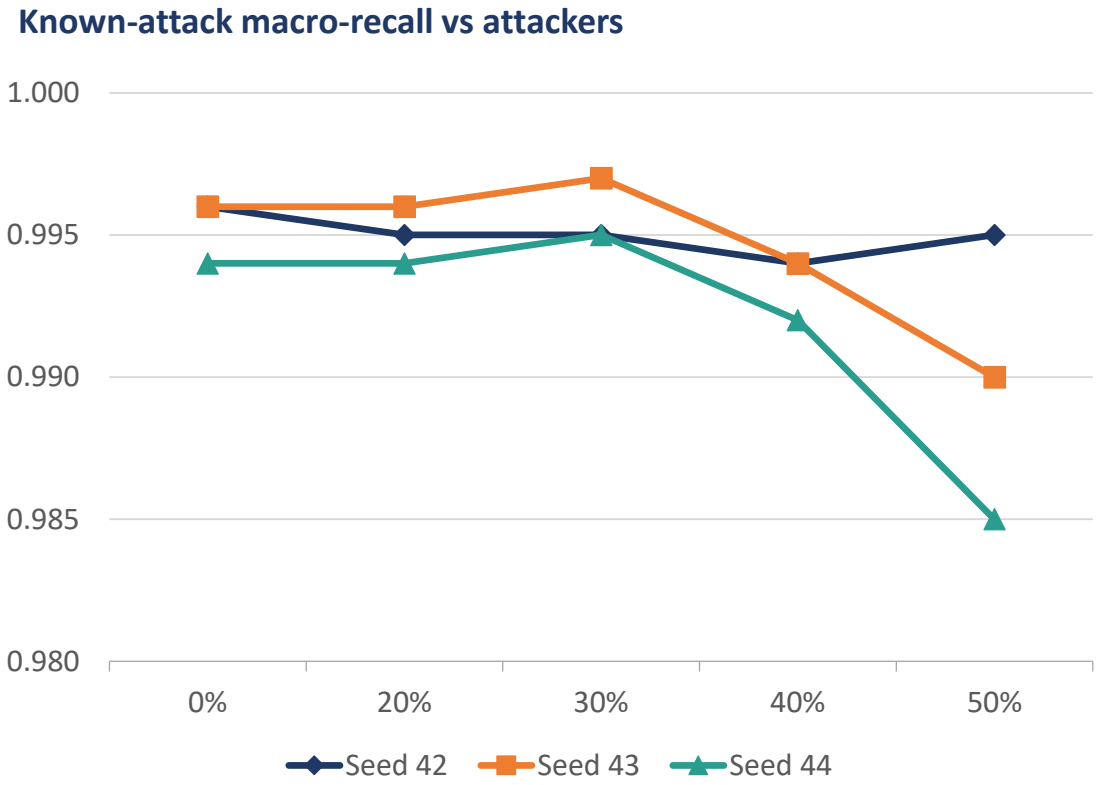


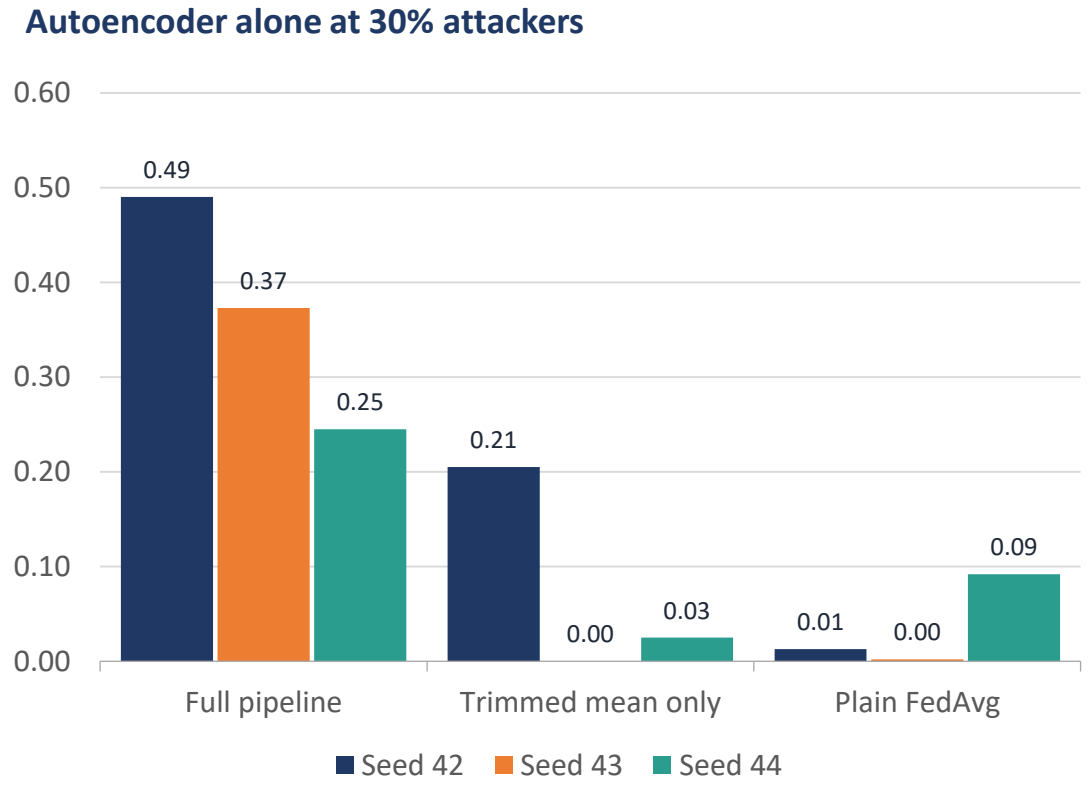
Each of the 14 attack classes was removed from all training in turn, then scored.

Per held-out class (mean of three seeds)



Results: Resistance to Poisoned Clients

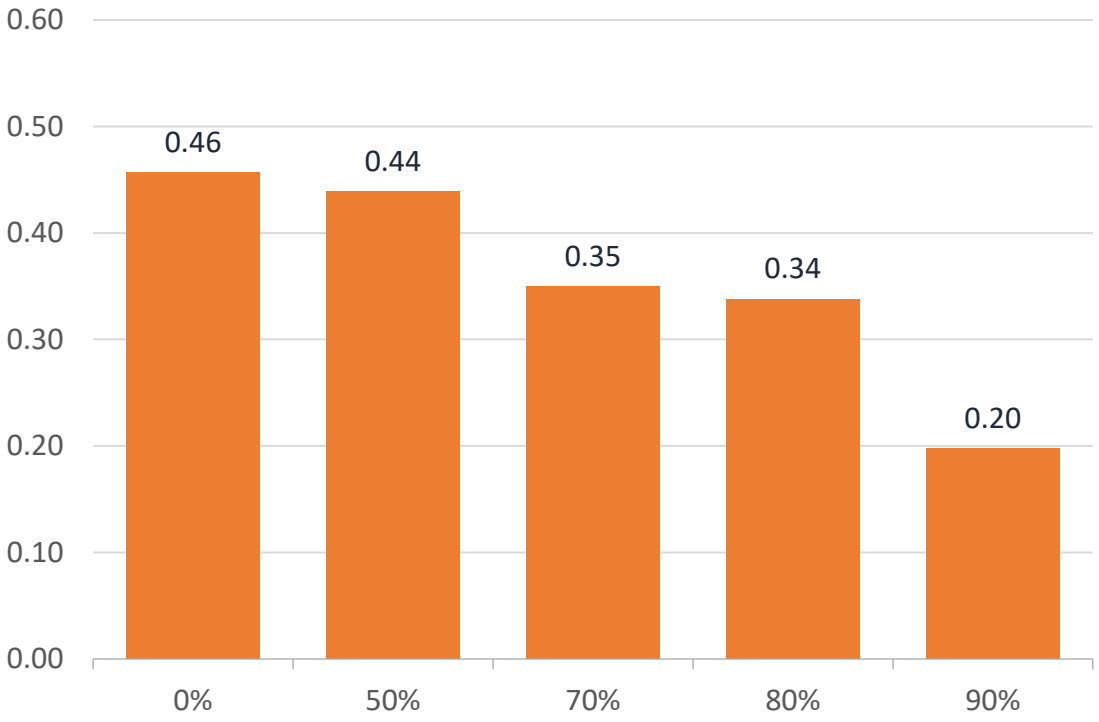




Known attacks stay at 0.985 to 0.997 recall up to 50% attackers. The trust filter protects the autoencoder: at 30% attackers its recall alone is 0.245 to 0.490, against 0.002 to 0.092 for plain FedAvg. The filter lets more attackers through than at 60,000 rows, and at 50% its median reference fails.

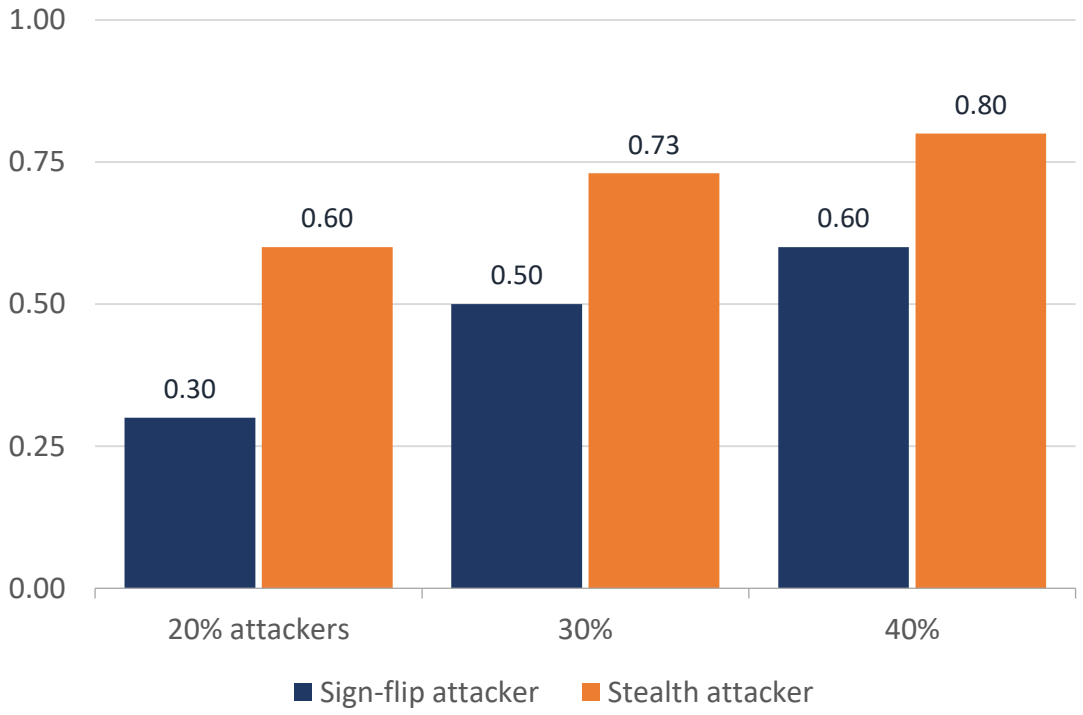
Results: Pruning and an Adaptive Attacker

Autoencoder recall alone vs pruning



Share of weights set to zero. Cascade macro-recall stays at 0.993 or above at every level, but updates are still sent dense, so no communication saving is shown.

Attacker survival in the filter



Survival is the share of attacker updates kept by the filter. The white-box stealth attacker is mostly missed. Single seed (42).

Results: SDN Mitigation (Phase B)

```
[phase_b] stage 1/4 done: Load the model bundle in 0.1s
Phase A bundle saved_models/full/c10_s42: 10 clients -> 10 Mininet hosts
Controller + bridge up at http://127.0.0.1:8080 (OpenFlow port 6653)
[phase_b] stage 3/4 done: Network, replay, classify and block in 8.8s

Phase B done in 12.0s:
  h1 (client 0) -> Backdoor
  confidence 0.997, boosting stage, 24543/24914 packets non-benign
  action: block

Flow rule read back from the switch:
  priority=100,ip,nw_src=10.0.0.1 actions=drop
```

14 of 14

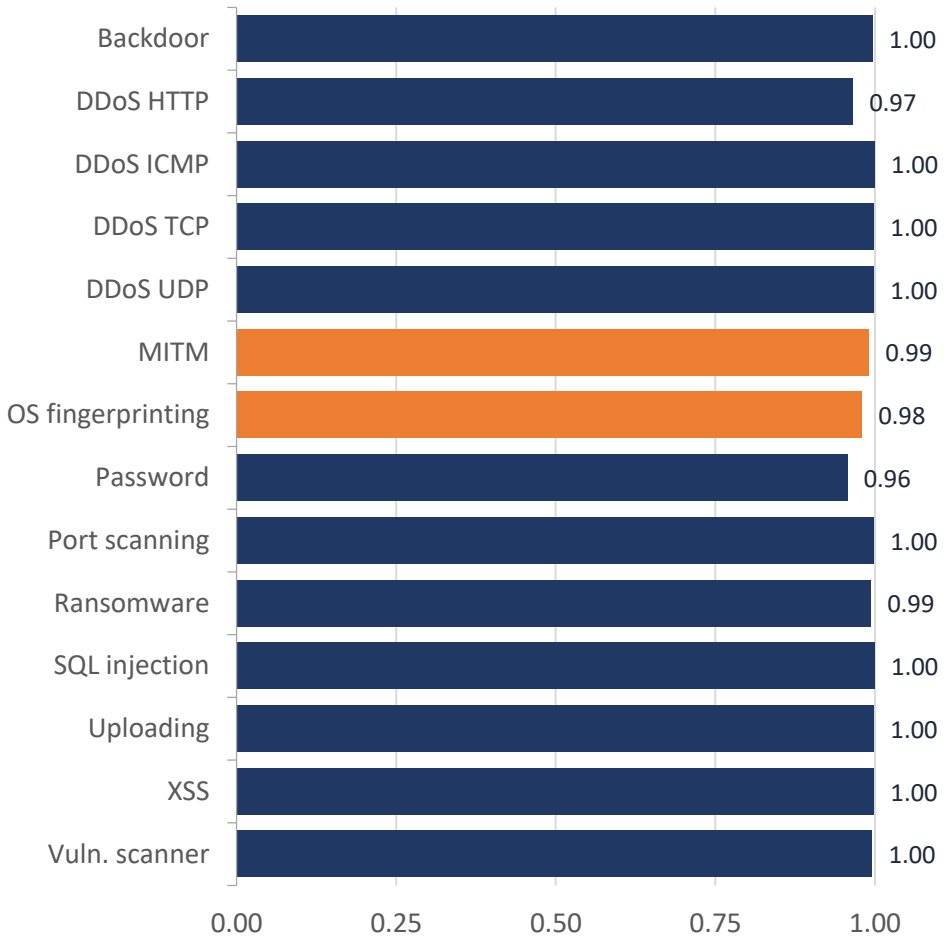
captures end in a block decision and a drop rule read back from the switch

12 of 14

classified as the right attack type

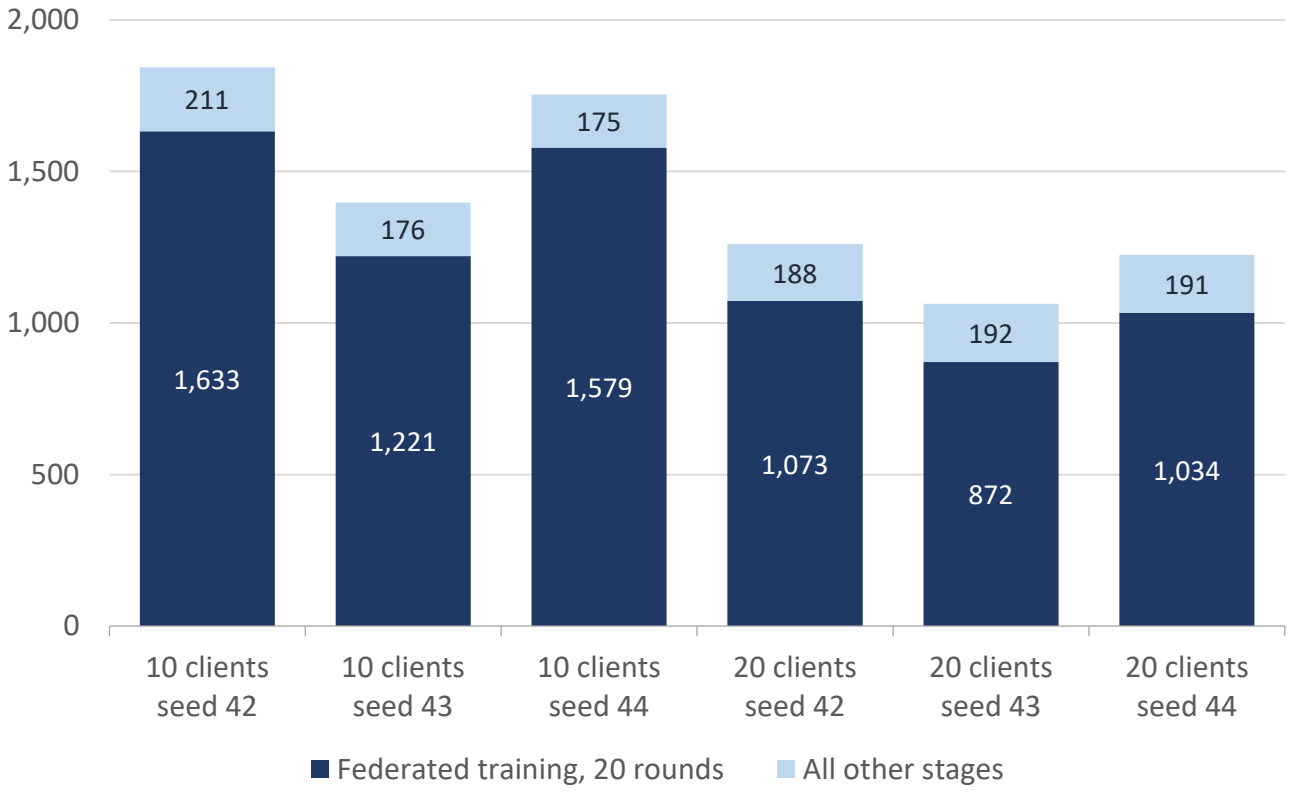
MITM was called Ransomware and OS fingerprinting DDoS ICMP; both were still blocked.

Confidence of the verdict per capture



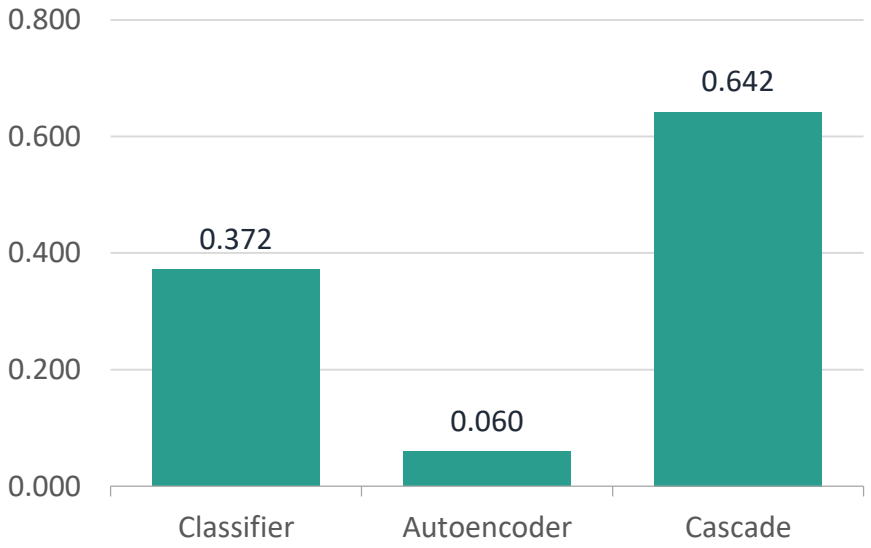
Results: Training Time and Decision Time

Time per full-dataset run (seconds)



Federated training is 82% to 90% of each run (18 to 31 minutes in total).

Median time to decide on one packet (ms)



3,760 packets/s

cascade throughput in batch mode, one CPU thread; model time only

Conclusion

Objective	What the measurements show
1. Malicious-client filtering	Known-attack recall 0.985 to 0.997 up to 50% sign-flip clients. At 30% the filtered autoencoder keeps 0.245 to 0.490 recall; plain FedAvg falls to 0.002 to 0.092. A stealth attacker is mostly missed.
2. Efficient, private FL and enforcement	Clients send only weights (about 39 kB) and histograms. Runs take 18 to 31 minutes on 1.89 million rows. All 14 captures blocked.
3. Pruning	Autoencoder recall falls from 0.457 to 0.350 at 70% sparsity; cascade recall stays at 0.993 or above. No communication saving yet.
4. Anomaly-based detection	Unseen attack classes: cascade 0.895 to 0.944, classifier alone 0.723 to 0.757, autoencoder alone 0.590 to 0.728.

Beyond the objectives: the classifier reaches 99.72% accuracy and 0.983 macro F1 on unseen connections, and the models decide on a packet in a median 0.642 ms. Limits: mitigation is per capture rather than inline, and the trust filter is weaker at full scale than on the 60,000-row pool.

Future Enhancements

1

Streaming mitigation

Classify flows after the first few packets so a rule is installed while an attack is still running.

2

Stronger defence

Defend against the white-box stealth attacker, for example with a small trusted root dataset.

3

Full-scale repeats

Explain why the filter lets more attackers through at full scale; repeat the stealth, pruning and clipping tests over more seeds.

4

Second dataset

Validate on traffic that includes benign web use, to test how much the service-port feature depends on the testbed.

5

Sparse updates

Send pruned updates to save communication, and test zero-day detection of pruned models.

6

Real hardware

Federate the classifier, then deploy on edge devices and a hardware OpenFlow switch.

Papers Published

No paper has been published yet.

A research paper describing the cascade, the trust filter and the connection-level evaluation is in preparation.

- Status: writing in progress (November 2026 onwards).
- Material for the paper: the repaired MITM class, the connection-held-out evaluation and the full-dataset federated results.

References

[1] M. A. Ferrag et al., “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning,” IEEE Access, vol. 10, pp. 40281–40306, 2022.

[2] V. Mothukuri et al., “Federated-learning-based anomaly detection for IoT security attacks,” IEEE Internet of Things Journal, vol. 9, no. 4, pp. 2545–2554, 2022.

[3] B. Li et al., “DeepFed: Federated deep learning for intrusion detection in industrial cyber-physical systems,” IEEE Trans. Industrial Informatics, vol. 17, no. 8, pp. 5615–5624, 2021.

[4] H. Zhang et al., “A two-stage intrusion detection method based on light gradient boosting machine and autoencoder,” Math. Biosciences and Engineering, vol. 20, no. 4, pp. 6966–6992, 2023.

[5] Z. Abou El Houda, A. S. Hafid, L. Khoukhi, “MiTFed: A privacy preserving collaborative network attack mitigation framework based on federated learning using SDN and blockchain,” IEEE Trans. Network Science and Engineering, vol. 10, no. 4, 2023.

[6] J. L. Hernandez-Ramos et al., “Intrusion detection based on federated learning: A systematic review,” ACM Computing Surveys, vol. 57, no. 12, 2025.

[7] X. Cao, M. Fang, J. Liu, N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” NDSS, 2021.

[8] D. Yin, Y. Chen, K. Ramchandran, P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” ICML, PMLR 80, pp. 5650–5659, 2018.

[9] M. M. Galal, “Provenance, not behaviour: A serialisation artifact in Edge-IIoTset,” arXiv:2608.15761, 2026.

Thank You

Questions?

Results and report: aronnnn1.github.io/FL-IDS-showcase